CSA02 C Programming – CO3 Assignment Answers

Assignment 3: Searching, Sorting, Merging and Arrays

Note: These are model answers based on the uploaded assignment sheets. Replace student-specific details and add your own compiler screenshots, timing measurements and GitHub links before submission.

Q1 – Searching

Linear search: best 1, worst n, average (n+1)/2 successful comparisons; unsuccessful n. Binary search requires sorted data and halves the interval: O(log n), at most about 17 comparisons for 100,000 entries. mid=low+(high-low)/2 avoids overflow. Hash table average O(1), worst O(n), extra O(n) space; interpolation average O(log log n) for uniform keys, worst O(n).

Q2 – Sorting

Bubble sort with early exit: O(n) best, O(n²) average/worst; stable. Selection sort O(n²) always, at most n–1 swaps, usually unstable. Insertion sort O(n) best/nearly sorted and O(n²) worst; stable. For [6,2,5,1,4,3], insertion passes end in [2,6,5,1,4,3], [2,5,6,1,4,3], [1,2,5,6,4,3], [1,2,4,5,6,3], [1,2,3,4,5,6]. Insertion sort best for nearly sorted data with 20 late entries. Counting sort O(n+k) time, O(k) space when key range k is small.

Q3 – Merge merit lists

```
int merge(const int a[],int na,const int b[],int nb,int out[]){int i=0,j=0,k=0;while(i<na&&j<nb)out[k++]=(a[i]>=b[j])?a[i++]:b[j
```

Output: 98 95 91 91 85 80 72. Time O(m+n), output space O(m+n). Each element copied once; comparison advances one pointer. Concatenate+qsort costs O((m+n)log(m+n)); simple in-place merging may cost O(mn) shifts.

Q4 – Rainfall arrays

For rain[38][12], annual total per district is a row sum; monthly state average is each column sum/38; wettest district-month is maximum over 456 entries. Deficient districts are those whose annual average is below 60% of the state average, applying the assignment's stated average definition consistently. Time O(38×12), extra O(1). Row-major address: base + (i×12+j)×sizeof(double). Loop j inside i for contiguous memory. A 3-D array rain[years][districts][months] is contiguous in row-major order.

Q5 – SMS text processing

```
int word_count(const char s[]){int count=0,inside=0;for(int i=0;s[i]!='\\0';i++){if(s[i]!=' '&&s[i]!='\\t'&&s[i]!='\\n'){if(!ins
```

Null terminator marks string end. Count words only on transition from whitespace to non-whitespace. Cache length rather than recomputing it each iteration. char s[] is a modifiable array; char *s points to a string literal and must not be modified. strcpy can overflow; use bounded input fgets and check lengths. gets was removed because it cannot limit input.

Q6 – Hostel name directory

Read each name with fgets into names[500][30], strip newline, sort with strcmp and swap using a temporary buffer. Binary search is O(log n) comparisons after sorting; duplicates become adjacent, so scan equal neighbors. strcmp compares byte values and is case-sensitive; apple and Apple may differ in ordering. Case-insensitive comparison can lowercase unsigned-char values before comparison.

Swapping two 30-byte names copies up to 60 characters; sorting an index array instead swaps small integers and preserves original names. Index sorting costs O(n log n) comparisons plus O(n) index space, but adds one indirection when displaying names. A char *list[500] pointer array similarly sorts pointers with qsort; each comparator may inspect up to L characters, giving O(n log n × L) character work and O(n) pointer space.

Q7 – Election count

Inputs: candidate count c≤12, vote count v≤5000, candidate names and vote numbers. Maintain int count[12], valid and invalid totals. For each vote x, if 1≤x≤c increment count[x–1] and valid; otherwise increment invalid. Sort candidate index/result records by count descending, detect tie when top two counts equal. One-pass counting is O(v), then sort O(c log c), space O(c).

```
typedef struct{int id,votes;} Result;\nvoid count_votes(const int votes[],int v,int c,int counts[],int *valid,int *invalid){*val
```

Count array beats searching candidate names for every vote (O(v·c)); sorting all votes costs O(v log v). For 500 candidates use a dynamically allocated count array or hash map; memory O(c). Test invalid 0, negative, c+1 and a tie.